Polymodels Hub: AI for Reaction Mechanism Discovery Challenge

I. The brief

Process development is the least visible and most expensive bottleneck between a molecule that works and a medicine a patient can take. Establishing what your reaction actually does, and proving it to a regulator, routinely runs to hundreds of experiments over many months. Every month of it burns patent life and delays patients.

Two things make that expensive in a way you may not have met yet:

1. Material is scarce. At this stage you might have a few grams of API, made over several weeks, and each experiment uses some of it. Screening broadly is not an option, so the experiments you do run have to be picked carefully.

2. You have to be able to explain yourself. Under Quality by Design you are expected to know why the process works, which parameters matter and what happens when they drift. Nobody expects a complete picture of the reaction network, but where the understanding is missing you make up for it elsewhere: extra pilot-scale runs to demonstrate the impurity stays controlled, narrower registered operating ranges, and less freedom to change anything later without going back to the regulator.

Understanding the mechanism pays off in any industry, and it pays unusually well in this one. It is how you improve yield with some confidence that the improvement will hold, and it is the only reliable way to control an impurity, because the right fix depends on where that impurity came from. If it forms from your product, a shorter hold helps. If it forms by a competing route off your starting material, a shorter hold makes it worse. Yield data alone cannot separate those two, and neither can conditions tuned by trial and error once the step moves from 5 mL to 500 L.

So the central question of pharmaceutical process development is not "what conditions give the best yield." It is **which reaction network is operating, with what kinetics, and what is the smallest set of experiments that would establish it?**

That question has a large and active literature, from classical model-discrimination design criteria through Bayesian experimental design to very recent work using language models to propose mechanisms, like SKAI. How it is approached varies a good deal between groups, and the method you choose can make a real difference to how many experiments a programme ends up needing.

That is the whole of this challenge. You are being scored on working out what is real, and on proposing the experiments that would settle it, not what the better operating point is.

II. The problem

You have joined process development at **Marco Therapeutics**, working on the final step of **MT-419**, an API in early development, heading into its first GMP campaign.

The step couples two intermediates:

A + B → P (your API)

It also produces an impurity, **Q**, which is genotoxic. Because it is genotoxic, the specification is not a target to trade against yield. It is a limit:

Q ≤ 15 ppm w/w relative to P in the isolated API

Nobody in the company can currently explain where **Q** comes from. Analytical development has one working hypothesis, chemistry has another, and the two imply completely different control strategies. Your job is to settle it.

III. What you are given

All the data ships with the repository in `data/`. There is no simulator to query and no experiments to buy during the event. What you have is what you have, which is exactly the position you would be in on a real programme.

All of it was collected on small-scale equipment: sealed, inerted 5 mL batch vessels held at a single temperature, sampled through an automated platform, and quantified against an internal standard. Main components are measured by UPLC-UV. Q sits close to the limit of quantitation and needs a separate, more sensitive UPLC-MS method, which is why it was assayed at only some of the timepoints in the data you have. Any experiment you propose later will be run on the same rig, with that trace method now automated onto every sample, so a timepoint you pay for returns Q as well.

You are given two reaction systems:

- System A, the reference system. A different, fully characterised reaction. The mechanism is disclosed in `data/system_A/MECHANISM.md`, and every species was measured, across five sets of conditions, with complete concentration-time profiles. System A exists so that you can build and validate your entire workflow against a known answer before pointing it at something unknown. If your method cannot recover the mechanism of System A, it will not recover anything.

- System B, the blind system. The **MT-419** step, and the one you are scored on. The mechanism is not disclosed. You get concentration-time data across several sets of conditions, all of them run on the same charge with A in four-fold excess, the ratio carried over from the discovery route. There are three complications.

    - At least one species is never measured. It does not appear in the data files at all. You will have to infer that it exists, and what it does, from its effect on the species you can see.

    - Measurements carry realistic noise, and trace-level **Q** measurements are noisier in relative terms than main-component assays.

    - Below the limit of quantitation you get a censored result rather than a number. A censored measurement is still information if you treat it as one.

The true mechanism of System B is one of the six candidates below. This is a closed-book problem: you are choosing between six named possibilities, not inventing chemistry from scratch.

The six candidates are provided as implemented ODE systems in `src/candidates/`, so you do not need to derive rate equations or write an integrator. Each carries unknown rate constants and activation energies which you must estimate.

Every rate constant in every candidate is Arrhenius. To keep the fitting well conditioned, each reaction is parameterised by its rate constant at a reference temperature of 75 °C together with an activation energy, rather than by a pre-exponential factor. Fitting a pre-exponential directly is possible, but it is badly scaled and it will cost you an afternoon.

	Network	**In words**
M1	A + B → P ; A + B → Q	Impurity forms directly from the reactants, parallel to product
M2	A + B → I ; I → P ; I → Q	A reactive intermediate that branches to product or impurity
M3	A + B → P ; P → Q	Impurity forms by degradation of the product, in series
M4	A + B → I ; I → P ; I + A → Q	Intermediate, with a second-order impurity route consuming more A
M5	A + B → P ; A + P → Q	Product reacts further with A to give the impurity
M6	A → I ; I + B → P ; I → Q	A activates first, and the activated species branches

A few things are worth flagging up front, so you know they are deliberate rather than mistakes in the data.

1. Several of these fit the System B data to within its noise. More than one will give you an excellent fit. That is the interesting part rather than a flaw in the data.

2. They imply different control strategies. M3 says control your hold time and downstream temperature. M4 says control your stoichiometry. M5 says do not let product and **A** coexist at high conversion. Choosing the wrong one means controlling the wrong parameter, which is riskier than controlling nothing, because it comes with false reassurance.

3. A flexible regression will not get you very far on its own. Fit a neural network or a spline to the observable species and you get a lovely fit and little else: no mechanism, no control strategy, and no way to design an experiment that discriminates. Whatever you build needs to be able to tell M1 from M4.

Finally, there is the campaign you would run next. You design it and submit it, but it is not run during the event and no results come back to you, so nothing you propose here will inform the rest of your submission. It is marked on how much it would have told you. The limits it has to fit inside are material rather than arbitrary, because material is what actually constrains you at this stage of development.

Constraint	Limit
Number of experiments	3 to 10
Total **A** consumed across the campaign	5.0 g
Total analytical samples	60

Every experiment is a 5 mL batch run on the equipment described above, held at a single temperature for the whole run. **A** has a molecular weight of 250 g/mol, so an experiment consumes 5 mL times its initial concentration of **A** times that molecular weight: a run at 1.0 M costs 1.25 g, a run at 0.25 M costs 0.31 g. The budget will not stretch to a broad screen, so you have to decide which conditions are worth paying for and how much of your sampling to spend on each.

One analytical sample is one timepoint, quantified for all observable species. Sixty samples across ten experiments is six timepoints each, so you will have to decide where in each run those timepoints earn their place.

Within an experiment you may set only the following:

- Temperature, from 50 to 120 °C, held constant for the whole run.
- Initial concentration of **A**, from 0.1 to 1.5 M.
- Initial concentration of **B**, from 0.1 to 1.5 M, set independently of **A**.
- Reaction duration up to 16 hours, and the times at which you sample.

That is the whole of it. The vessel is 5 mL, and the solvent, the base and the catalyst system are fixed. No new reagents, no temperature ramps, and no analytical method that sees a species the released data does not already contain.

Design the campaign as a single block. Every experiment in it is committed at the same time, so you cannot make the fourth depend on what the third would have told you.

One thing worth sitting with before you spend the budget: an experiment at conditions close to data you already have will tell you very little, however carefully you sample it.

IV. What you need to submit

Everything lives in your team's repository. There are four deliverables.

1. A ranked hypothesis set, in `submission/hypotheses.yaml`: the six candidates in rank order, with a posterior probability or equivalent weight for each, and a short justification. Ranking honestly matters more than ranking confidently. If two candidates are genuinely inseparable on the data you were given, say so and weight them accordingly. You will score better for saying so than for picking one and hoping.

2. A calibrated prediction. `just predict` writes predictions for a held-out set of conditions, `data/heldout_conditions.csv`, which were not in your training data. Some of these conditions lie outside the range the released data covers, so expect to be extrapolating rather than interpolating, and size your intervals accordingly. For each condition and each observable species, give a point prediction and a 90% prediction interval. The intervals are checked for calibration.

3. A confirmatory experimental campaign. `just suggest` writes `submission/campaign.yaml`: the experiments you would run next to discriminate between your surviving hypotheses, within the limits set out at the end of section III. For each proposed experiment, give the conditions, the sampling times, and which species you would assay.

4. A presentation. Every figure in it must be able to be replicated using `just replicate`. It must also include an honest account of the limitations of your approach: what your analysis cannot tell you, which of your assumptions you are least confident in, and what you would do next given more time or more material. We would far rather see a clear-eyed account of what you could not resolve than an overclaim.

V. The repository contract

Your submission is run by the judges, in this order, on a clean machine.

```
just install      # builds with no manual intervention
just predict      # writes submission/predictions.csv
just suggest      # writes submission/campaign.yaml
just replicate    # regenerates every figure in your presentation
```

This one is worth getting right early, because it is a gate rather than a scoring component: if the four commands do not run in sequence on a clean machine, we can only judge your submission on the presentation. Likewise, if a figure in your slides does not come out of `just replicate`, we cannot count it as evidence. Consider it a small contribution to solving the replication crisis.

The four recipes live in a `justfile` at the root of your repository. `just` is a single binary and it is already installed on the lab machines, or available through the usual package managers if you are on your own laptop.

Pin your dependencies, use nothing outside PyPI, require no network access at run time, and keep the total runtime under ten minutes. Test it by cloning your own repository into a fresh directory and running the four commands there.

VI. Approaches worth knowing about

There is no expected solution path and the marks do not reward one. `RESOURCES.md` sets out six different routes to a solution, with one or two key references under each, notes on which are open access, and a suggested order to work through them.

- Classical model-based design of experiments.
- Bayesian inference and model selection.
- Symbolic regression and automated model discovery.
- Gaussian processes and hybrid models.
- Training a classifier on simulated data.
- LLM-assisted discovery.

Read the one or two things under your chosen route properly rather than skimming all of them. Some of what already exists will do a large part of this challenge for you, which is allowed and sensible. Just remember the marks are in the reasoning rather than the output. Lean on a package without understanding what it did, and you can end up with a ranking you cannot defend, an interval you cannot explain, and a campaign designed by somebody else's objective function.

AI assistance is allowed and expected, and there is no penalty for using it. You will be asked about your own code and your own reasoning when you present, so do make sure you can talk through what you submit.

One last thing. This is a genuinely hard problem and it is meant to be. Nobody expects a tidy answer, and a team that is honest about what it could not resolve will do better here than a team that papers over it.

Good luck, and happy coding!